Chapter 12 Exception Handling and Text IO

Motivations

When a program runs into a runtime error, the program terminates abnormally. How can you handle the runtime error so that the program can continue to run or terminate gracefully? This is the subject we will introduce in this chapter.

Objectives

- To get an overview of exceptions and exception handling (§12.2).
- To explore the advantages of using exception handling (§12.2).
- To distinguish exception types: **Error** (fatal) vs. **Exception** (nonfatal) and checked vs. unchecked (§12.3).
- To declare exceptions in a method header (§12.4.1).
- To throw exceptions in a method (§12.4.2).
- To write a **try-catch** block to handle exceptions (§12.4.3).
- To explain how an exception is propagated (§12.4.3).
- To obtain information from an exception object (§12.4.4).
- To develop applications with exception handling (§12.4.5).
- To use the **finally** clause in a **try-catch** block (§12.5).
- To use exceptions only for unexpected errors (§12.6).
- To rethrow exceptions in a **catch** block (§12.7).
- To create chained exceptions (§12.8).
- To define custom exception classes (§12.9).
- To discover file/directory properties, to delete and rename files/directories, and to create directories using the **File** class (§12.10).
- To write data to a file using the **PrintWriter** class (§12.11.1).
- To use try-with-resources to ensure that the resources are closed automatically (§12.11.2).
- To read data from a file using the **Scanner** class (§12.11.3).
- To understand how data is read using a **Scanner** (§12.11.4).
- To develop a program that replaces text in a file (§12.11.5).
- To read data from the Web (§12.12).
- To develop a Web crawler (§12.13).

Exception

- *Exception handling enables a program to deal with exceptional situations and continue its normal execution.*
- *Runtime errors* occur while a program is running if the JVM detects an operation that is impossible to carry out.
- If you access an array using an index that is out of bounds, you will get a runtime error with an **ArrayIndexOutOfBoundsException**.
- If you enter a **double** value when your program expects an integer, you will get a runtime error with an **InputMismatchException**.

Exception

- An *exception* is an object that represents an error or a condition that prevents execution from proceeding normally. If the exception is not handled, the program will terminate abnormally.

Example

```
1  import java.util.Scanner;
2
3  public class Quotient {
4    public static void main(String[] args) {
5      Scanner input = new Scanner(System.in);
6
7      // Prompt the user to enter two integers
8      System.out.print("Enter two integers: ");
9      int number1 = input.nextInt();
10     int number2 = input.nextInt();
11
12     System.out.println(number1 + " / " + number2 + " is " +
13       (number1 / number2));
14   }
15 }
```

```
Enter two integers: 5 2 Enter
5 / 2 is 2
```

```
Enter two integers: 3 0 Enter
Exception in thread "main" java.lang.ArithmeticException: / by zero
at Quotient.main(Quotient.java:11)
```

Handling Exception

```
import java.util.Scanner;

public class QuotientWithException {
  public static int quotient(int number1, int number2) {
    if (number2 == 0)
      throw new ArithmeticException("Divisor cannot be zero");

    return number1 / number2;
  }

  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    // Prompt the user to enter two integers
    System.out.print("Enter two integers: ");
    int number1 = input.nextInt();
    int number2 = input.nextInt();

    try {
      int result = quotient(number1, number2);
      System.out.println(number1 + " / " + number2 + " is "
        + result);
    }
    catch (ArithmeticException ex) {
      System.out.println("Exception: an integer " +
        "cannot be divided by zero ");
    }

    System.out.println("Execution continues ...");
  }
}
```

If an ArithmeticException occurs

Handling Exception

```
import java.util.Scanner;

public class QuotientWithException {
  public static int quotient(int number1, int number2) {
    if (number2 == 0)
      throw new ArithmeticException("Divisor cannot be zero");

    return number1 / number2;
  }

  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    // Prompt the user to enter two integers
```

```
Enter two integers: 5 3 Enter
5 / 3 is 1
Execution continues ...
```

```
Enter two integers: 5 0 Enter
Exception: an integer cannot be divided by zero
Execution continues ...
```

```
    }

    System.out.println("Execution continues ...");
  }
}
```

Exception

- The value thrown, in this case **new ArithmeticException("Divisor cannot be zero")**, is called an *exception*. The execution of a **throw** statement is called *throwing an exception*.
- The exception is an object created from an exception class. In this case, the exception class is **java.lang.ArithmeticException**. The constructor **ArithmeticException(str)** is invoked to construct an exception object, where **str** is a message that describes the exception.
- The **try** block contains the code that is executed in normal circumstances. The exception is caught by the **catch** block to *handle the exception*.

Exception

- The identifier **ex** in the **catch**–block header **catch**(ArithmeticException ex)

  acts very much like a parameter in a method. Thus, this parameter is referred to as a **catch** block parameter.

- The type (e.g., **ArithmeticException**) preceding **ex** specifies what kind of exception the **catch** block can catch.

- Once the exception is caught, you can access the thrown value from this parameter in the body of a **catch** block.

Handling InputMismatchException

```
import java.util.*;

public class InputMismatchExceptionDemo {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    boolean continueInput = true;

    do {
      try {
        System.out.print("Enter an integer: ");
        int number = input.nextInt();

        // Display the result
        System.out.println(
          "The number entered is " + number);

        continueInput = false;
      }
      catch (InputMismatchException ex) {
        System.out.println("Try again. (" +
          "Incorrect input: an integer is required)");
        input.nextLine(); // Discard input
      }
    } while (continueInput);
  }
}
```

If an InputMismatchException occurs

Handling InputMismatchException

```java
import java.util.*;

public class InputMismatchExceptionDemo {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    boolean continueInput = true;

    do {
      try {
        System.out.print("Enter an integer: ");
        int number = input.nextInt();

        // Display the result
        System.out.println(
          "The number entered is " + number);

        continueInput = false;
      }
      catch (InputMismatchException ex) {
        System.out.println("Try again. (" +
          "Incorrect input: an integer is required)");
        input.nextLine(); // Discard input
      }
    } while (continueInput);
  }
}
```

If an InputMismatchException occurs

```
Enter an integer: 3.5 Enter
Try again. (Incorrect input: an integer is required)
Enter an integer: 4 Enter
The number entered is 4
```

Remark

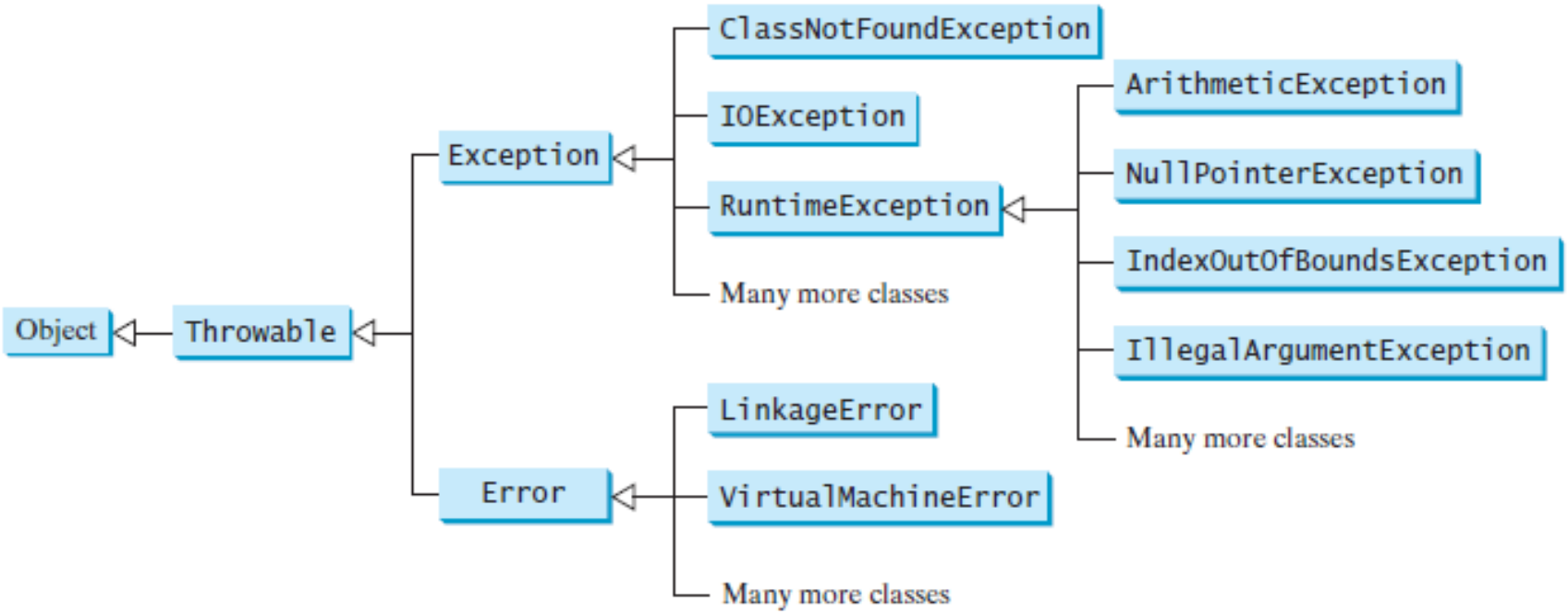
`catch` block catches :

ALL exceptions of its type and subclasses of its type

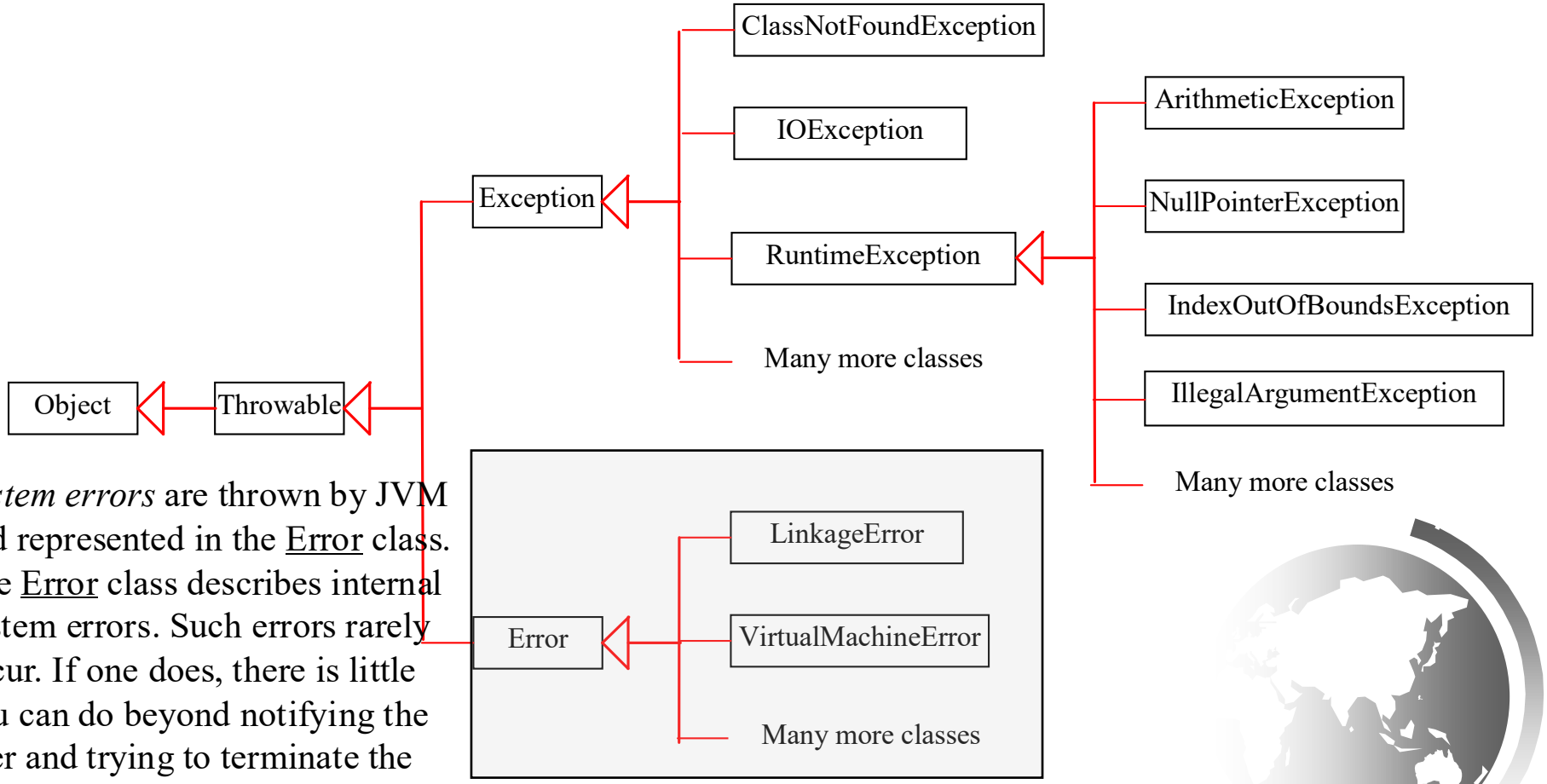
Exception

- Exceptions are objects, and objects are defined using classes. The root class for exceptions is **java.lang.Throwable**.
- The preceding section used the classes **ArithmeticException** and **InputMismatchException**
- There are many predefined exception classes in the Java API. All Java exception classes inherit directly or indirectly from **Throwable**. You can create your own exception classes by extending **Exception** or a subclass of **Exception**.

Exception Types



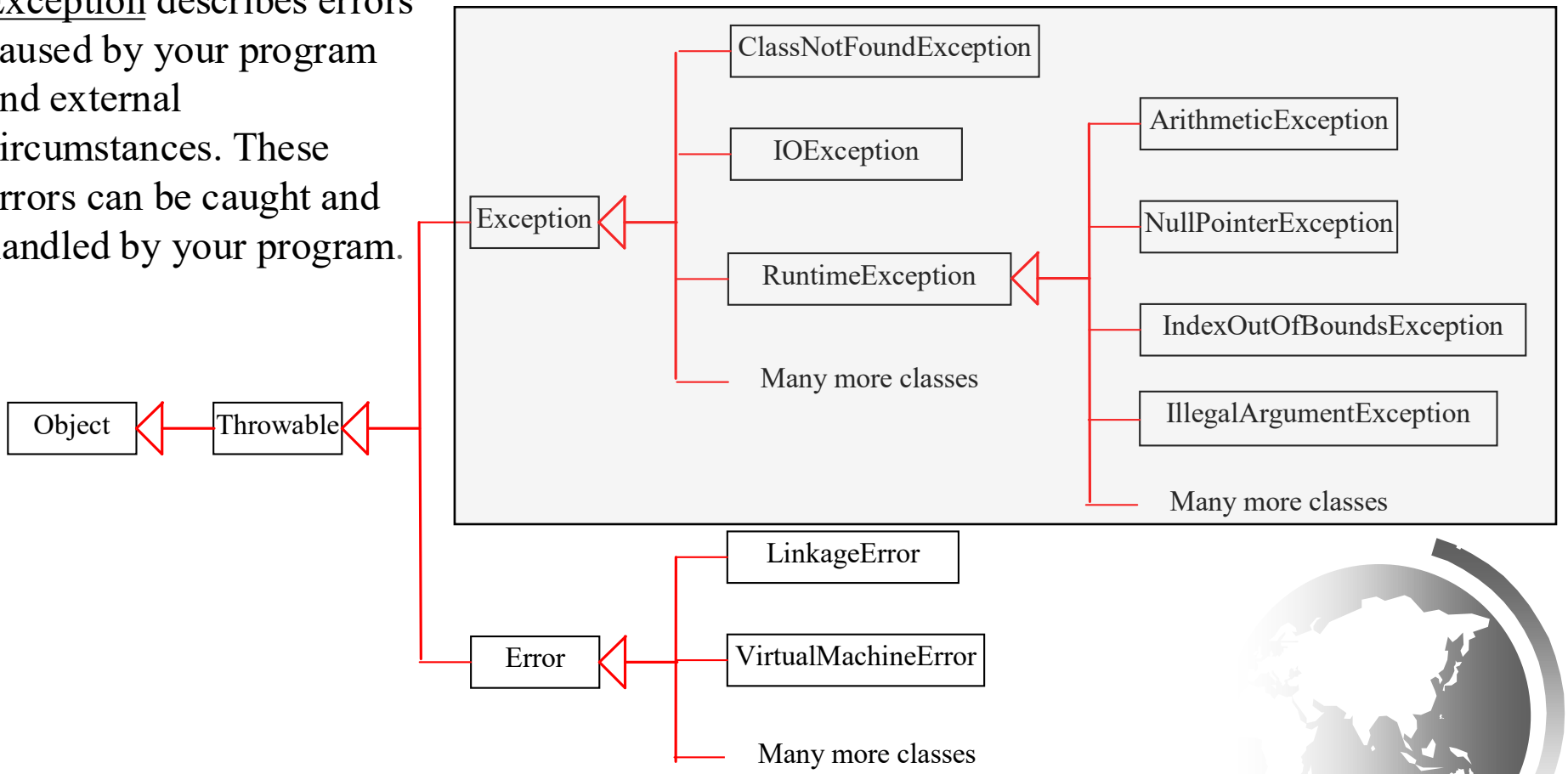
System Errors



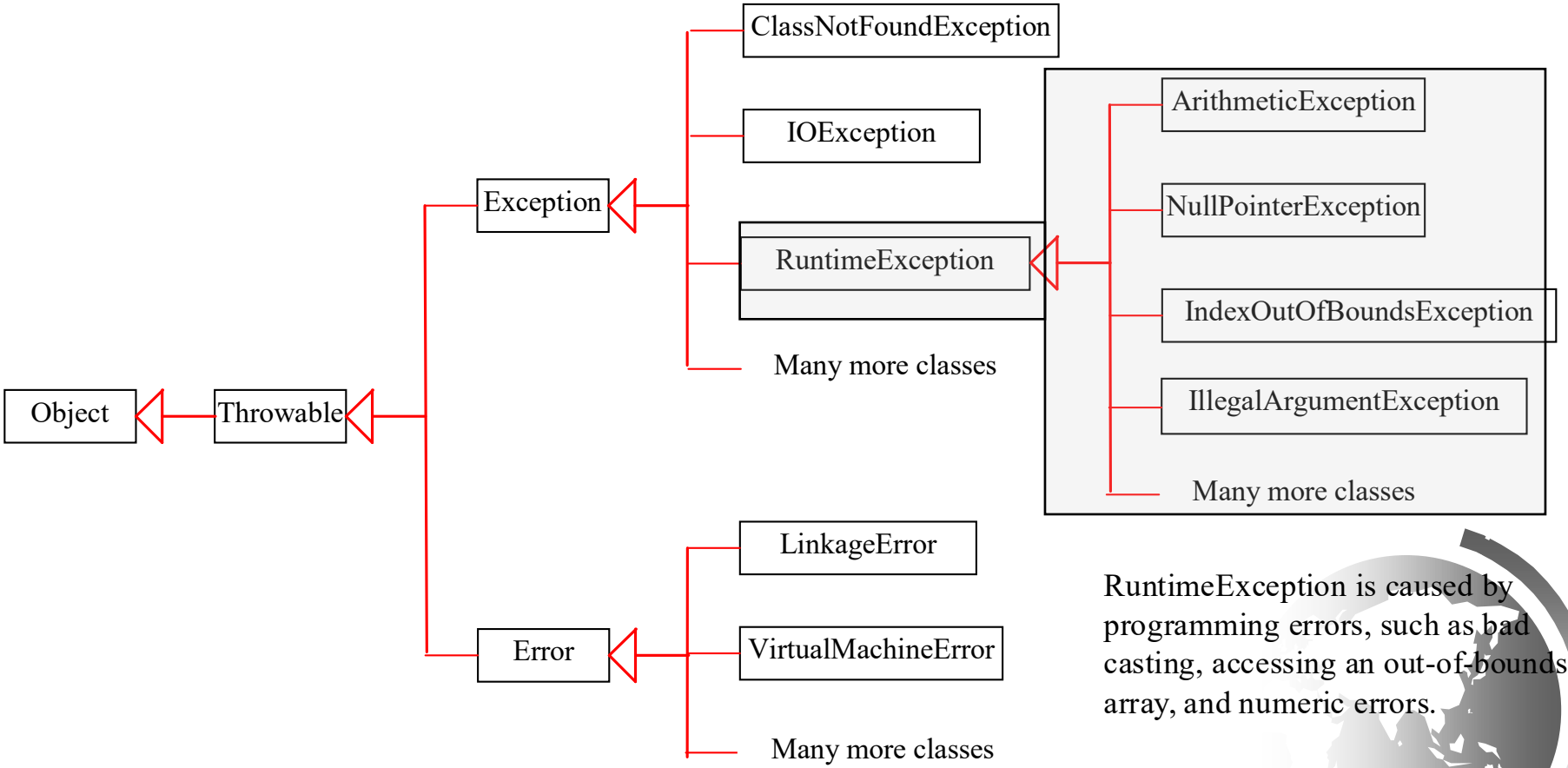
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Checked versus Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

Checked exceptions are checked at compile time. If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using *throws* keyword.

Checked versus Unchecked Exceptions

The program doesn't compile, because the function main() uses FileReader() that throws a checked exception named as *FileNotFoundException*

```
class Main {
    public static void main(String[] args) {
        FileReader file = new FileReader("C:\\test\\a.txt");
        BufferedReader fileInput = new BufferedReader(file);

        // Print first 3 lines of file "C:\test\a.txt"
        for (int counter = 0; counter < 3; counter++)
            System.out.println(fileInput.readLine());

        fileInput.close();
    }
}
```

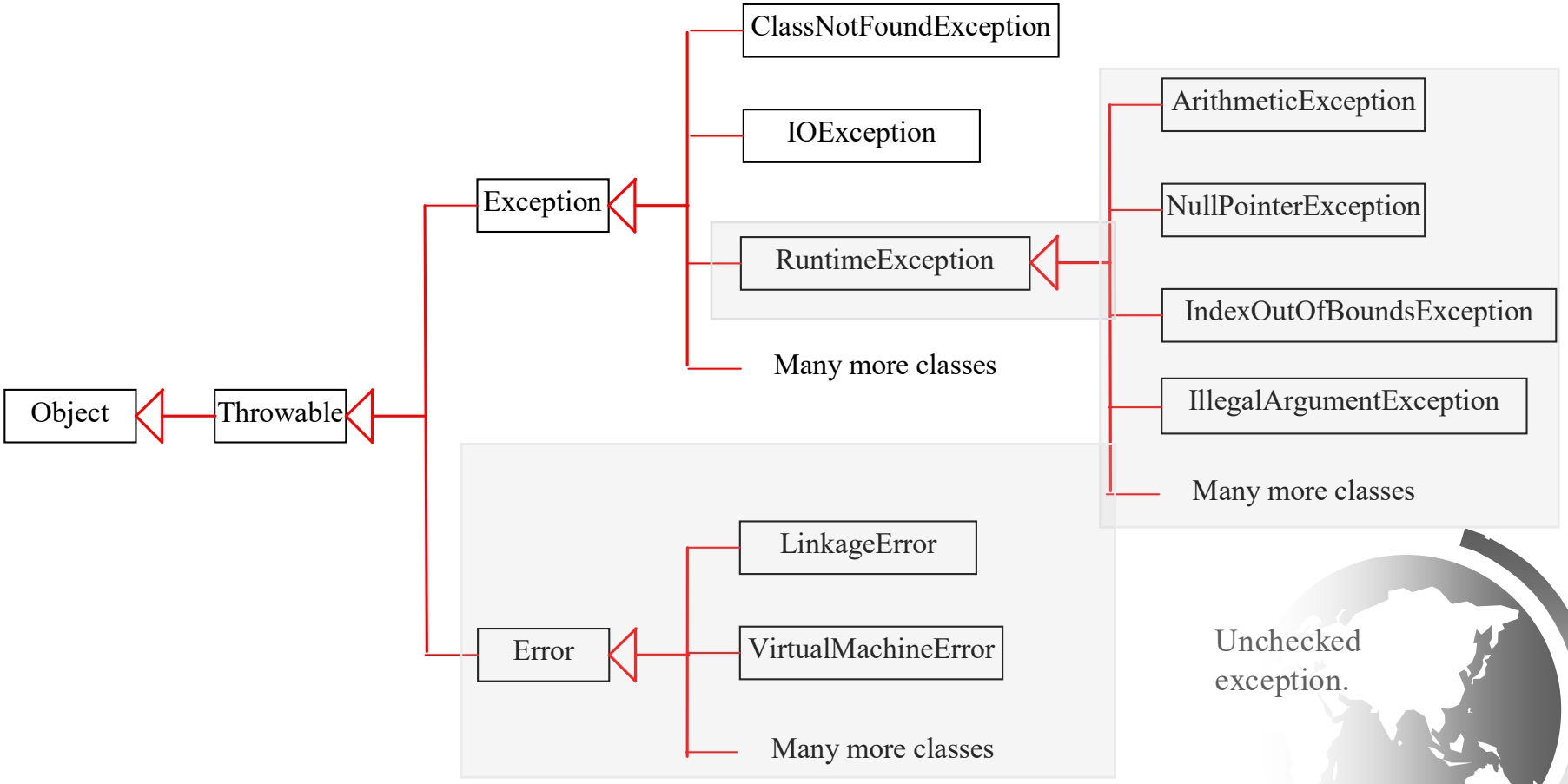
```
class Main {
    public static void main(String[] args) throws IOException {
        FileReader file = new FileReader("C:\\test\\a.txt");
        BufferedReader fileInput = new BufferedReader(file);

        // Print first 3 lines of file "C:\test\a.txt"
        for (int counter = 0; counter < 3; counter++)
            System.out.println(fileInput.readLine());

        fileInput.close();
    }
}
```

Unchecked Exceptions

In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example, a NullPointerException is thrown if you access an object through a reference variable before an object is assigned to it; an IndexOutOfBoundsException is thrown if you access an element in an array outside the bounds of the array. These are the logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in the program.

To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.

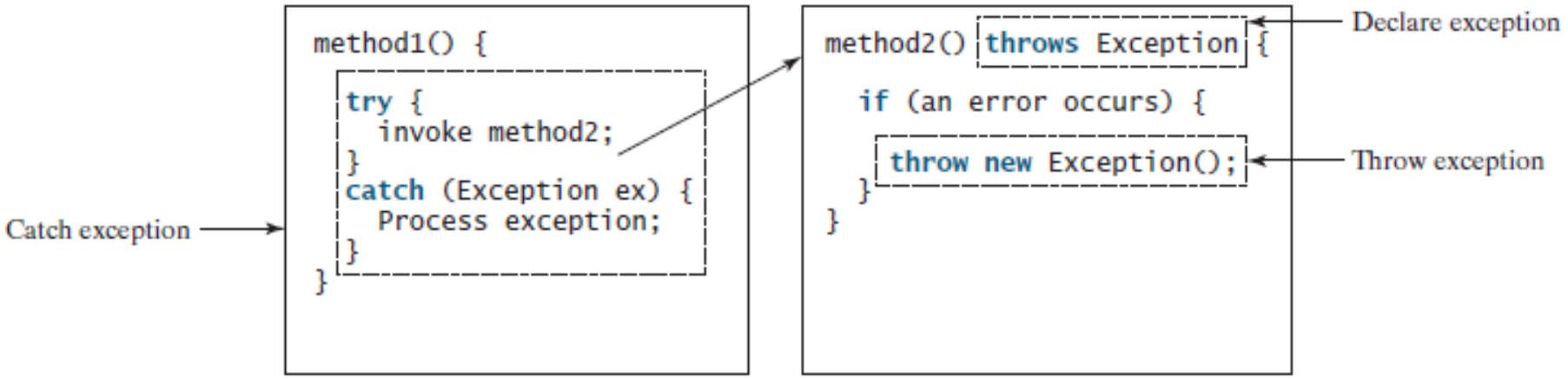
Unchecked Exceptions



Unchecked exception.

Declaring, Throwing, and Catching Exceptions

- Java's exception-handling model is based on three operations: *declaring an exception*, *throwing an exception*, and *catching an exception*, as shown below.



Declaring Exceptions

- Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*. Java does not require declaring **Error** and **RuntimeException** (unchecked exceptions) explicitly in the method. All others thrown by the method must be explicitly declared.

```
public void myMethod()
    throws IOException

public void myMethod()
    throws IOException, OtherException
```

Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();
throw ex;
```

Throwing Exceptions Example

```
public void setRadius(double newRadius)
      throws IllegalArgumentException {
    if (newRadius >= 0)
      radius =  newRadius;
    else
      throw new IllegalArgumentException(
        "Radius cannot be negative");
  }
```

- In general, each exception class in the Java API has at least two constructors: a no-arg constructor, and a constructor with a **String** argument that describes the exception. This argument is called the *exception message*, which can be obtained using **getMessage()**.
- The keyword to declare an exception is **throws**, and the keyword to throw an exception is **throw**.

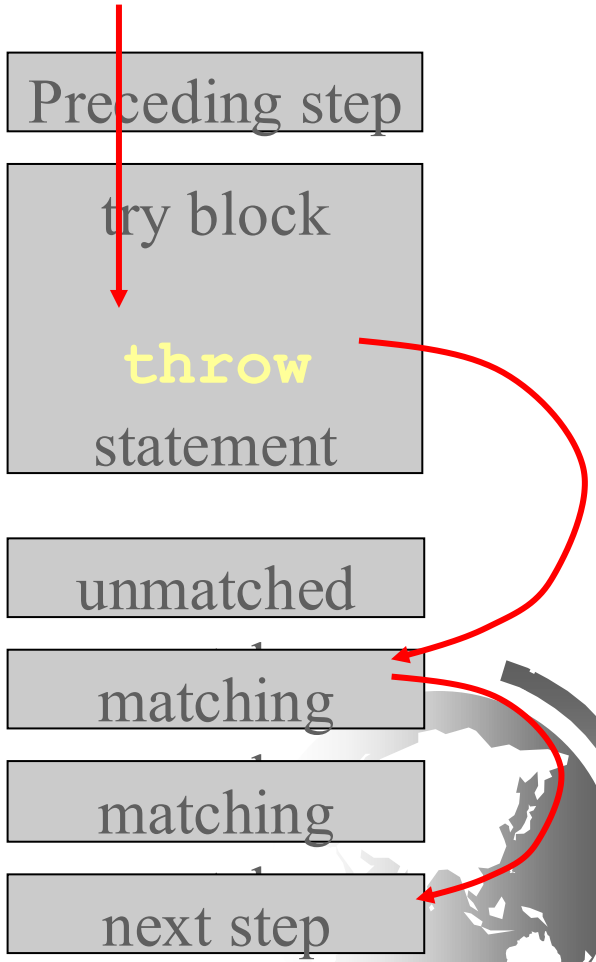
Catching Exceptions

```
try {
  statements;  // Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```

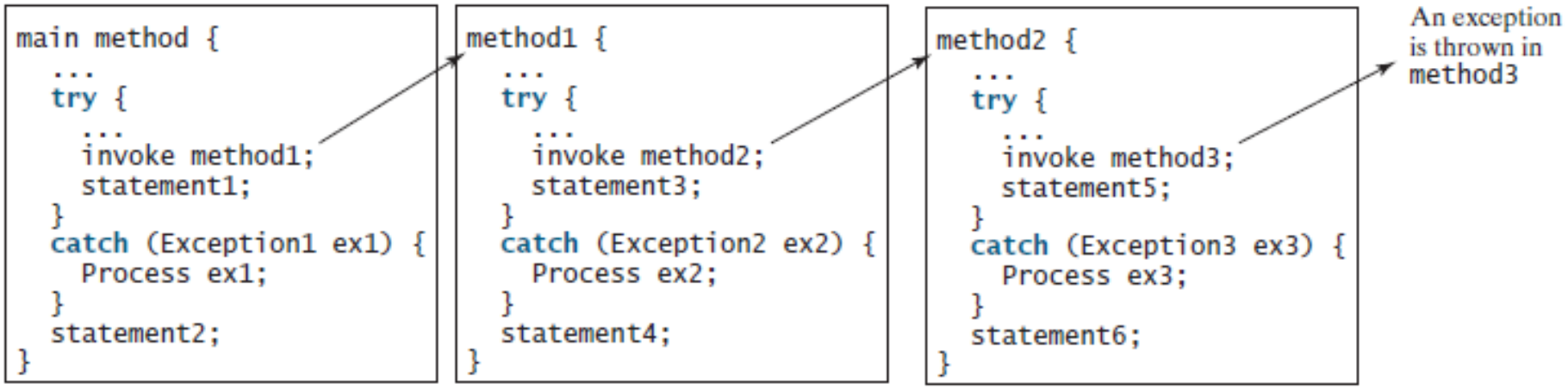
If one of the statements inside the try block throws an exception, Java skips the remaining statements in the try block and starts the process of finding the code to handle the exception.

Catching Exceptions

If there are multiple `catch` blocks that match a particular exception type, only the first matching `catch` block executes

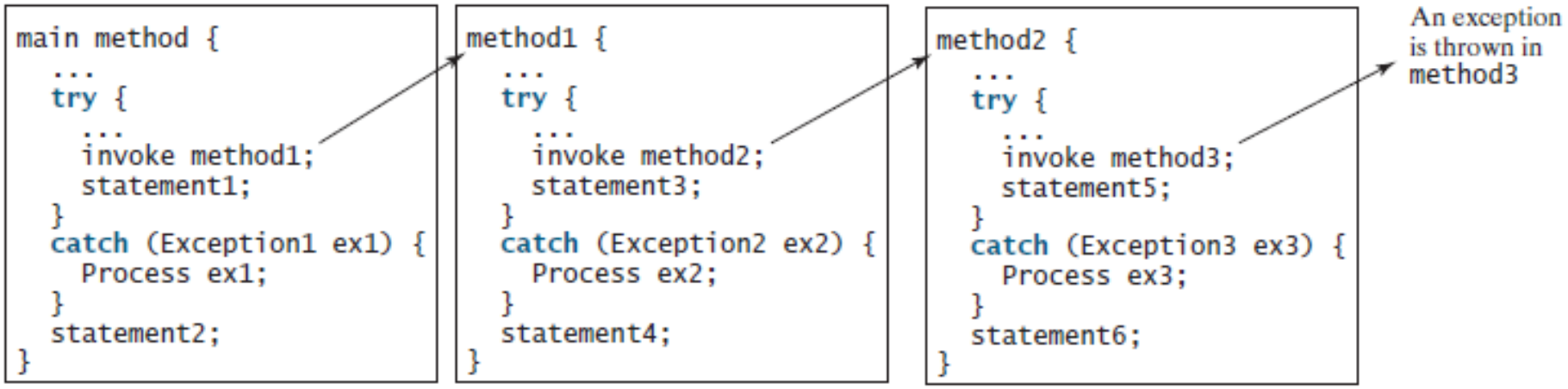


Catching Exceptions



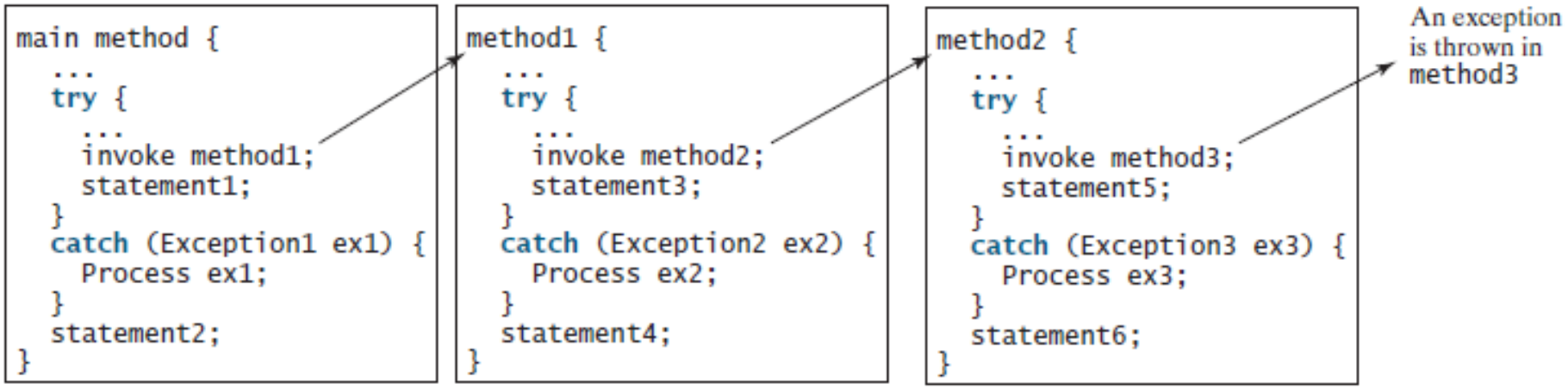
Suppose the **main** method invokes **method1**, **method1** invokes **method2**, **method2** invokes **method3**, and **method3** throws an exception, as shown above.

Catching Exceptions



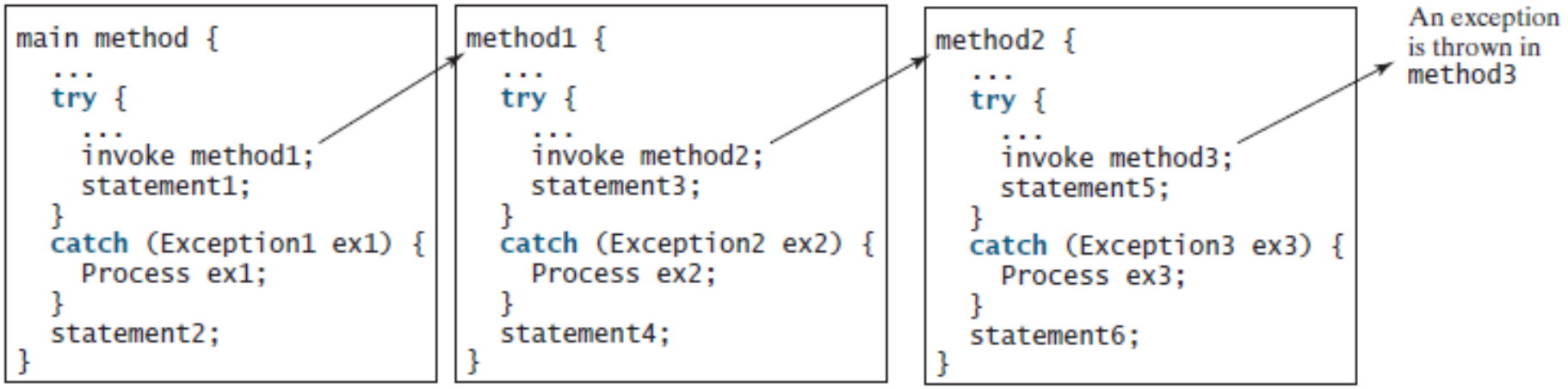
If the exception type is Exception3, it is caught by the catch block for handling exception ex3 in method2. statement5 is skipped, and statement6 is executed.

Catching Exceptions



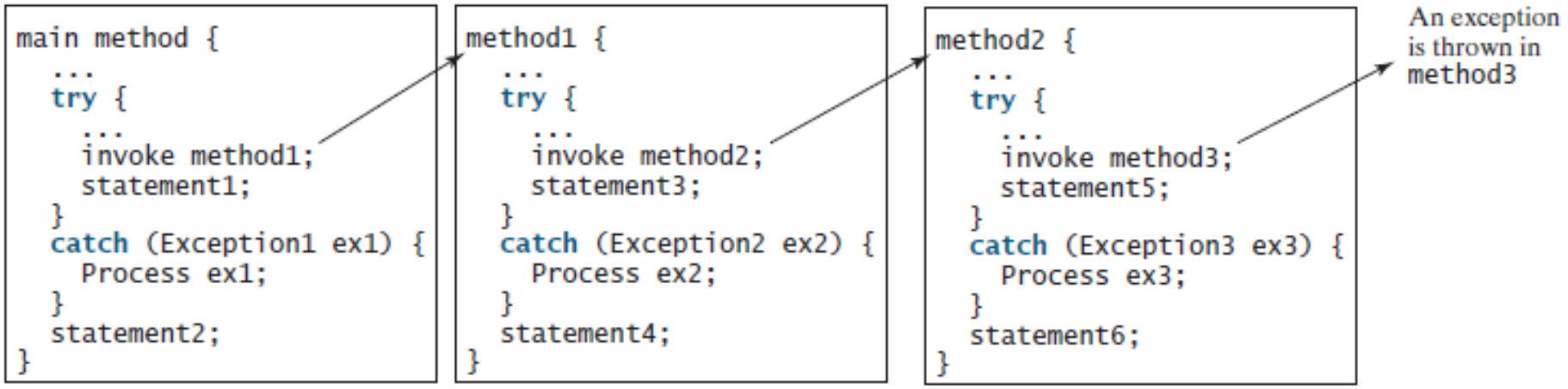
If the exception type is Exception2, method2 is aborted, the control is returned to method1, and the exception is caught by the catch block for handling exception ex2 in method1. statement3 is skipped, and statement4 is executed.

Catching Exceptions



If the exception type is Exception1, method1 is aborted, the control is returned to the main method, and the exception is caught by the catch block for handling exception ex1 in the main method. statement1 is skipped, and statement2 is executed.

Catching Exceptions



If the exception type is not caught in method2, method1, or main, the program terminates, and statement1 and statement2 are not executed.

Catch or Declare Checked Exceptions

Suppose p2 is defined as follows:

```
void p2() throws IOException {
  if (a file does not exist) {
     throw new IOException("File does not exist");
  }

  ...
}
```

Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method (`p2()` in previous slide) declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

Example

```
public class CircleWithException {
  /** The radius of the circle */
  private double radius;

  /** The number of the objects created */
  private static int numberOfObjects = 0;

  /** Construct a circle with radius 1 */
  public CircleWithException() {
    this(1.0);
  }

  /** Construct a circle with a specified radius */
  public CircleWithException(double newRadius) {
    setRadius(newRadius);
    numberOfObjects++;
  }

  /** Return radius */
  public double getRadius() {
    return radius;
  }

  /** Set a new radius */
  public void setRadius(double newRadius)
      throws IllegalArgumentException {
    if (newRadius >= 0)
      radius = newRadius;
    else
      throw new IllegalArgumentException(
        "Radius cannot be negative");
  }

  /** Return numberOfObjects */
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  public double findArea() {
    return radius * radius * 3.14159;
  }
}
```

Example

```
public class TestCircleWithException {
  public static void main(String[] args) {
    try {
      CircleWithException c1 = new CircleWithException(5);
      CircleWithException c2 = new CircleWithException(-5);
      CircleWithException c3 = new CircleWithException(0);
    }
    catch (IllegalArgumentException ex) {
      System.out.println(ex);
    }

    System.out.println("Number of objects created: " +
      CircleWithException.getNumberOfObjects());
  }
}
```

```
java.lang.IllegalArgumentException: Radius cannot be negative
Number of objects created: 1
```

Example

```
3
4
5  public class HelloWorld {
6          public static void main(String[] args)
7          {
8              try
9              {
10                 int a = 20/0;
11                 a = a + 1;
12             } catch (Exception e)
13             {
14
15                 // Prints what exception has been thrown
16                 System.out.println(e);
17
18                 System.out.println(e.getMessage());
19             }
20         }
21 }
22
23
24
25
```

Problems @ Javadoc Declaration Console Coverage Error

<terminated> HelloWorld [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0

java.lang.ArithmeticException: / by zero
/ by zero

Rethrowing Exceptions

Java allows an exception handler to rethrow the exception if the handler cannot process the exception or simply wants to let its caller be notified of the exception.

```
try {
  statements;
}
catch(TheException ex) {
  perform operations before exits;
  throw ex;
}
```

The statement **throw ex** rethrows the exception to the caller so that other handlers in the caller get a chance to process the exception **ex**.

The `finally` Clause

Occasionally, you may want some code to be executed regardless of whether an exception occurs or is caught. Java has a **finally** clause that can be used to accomplish this objective.

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}
```

Trace a Program Execution

Suppose no exceptions in the statements

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed

Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Next statement in the method is executed

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed.

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The next statement in the method is now executed.

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

statement2 throws an exception of type Exception2.

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Handling exception

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Execute the final block

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Rethrow the exception and control is transferred to the caller

Cautions When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify. Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

When to Throw Exceptions

- Suppose that an exception occurs in a method. If you want the exception to be processed by its caller, you should create an exception object and throw it. If you can handle the exception in the method where it occurs, there is no need to throw it.

When to Use Exceptions

- When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, consider the following code:

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```

When to Use Exceptions

The exception above is better replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

Defining Custom Exception Classes

- Use the exception classes in the API whenever possible.
- Define custom exception classes if the predefined classes are not sufficient.
- Define custom exception classes by extending Exception or a subclass of Exception.

Custom Exception Class Example

```
public class InvalidRadiusException extends Exception {
  private double radius;

  /** Construct an exception */
  public InvalidRadiusException(double radius) {
    super("Invalid radius " + radius);
    this.radius = radius;
  }

  /** Return the radius */
  public double getRadius() {
    return radius;
  }
}
```

This custom exception class extends **java.lang.Exception**. The **Exception** class extends **java.lang.Throwable**. All the methods (e.g., **getMessage()**, **toString()**, and **printStackTrace()**) in **Exception** are inherited from **Throwable**. The **Exception** class contains four constructors. Among them, Exception() and Exception(String message) are often used.

Custom Exception Class Example

```
class CircleWithCustomException {
  /** The radius of the circle */
  private double radius;

  /** The number of objects created */
  private static int numberOfObjects = 0;

  /** Construct a circle with radius 1 */
  public CircleWithCustomException() throws InvalidRadiusException {
    this(1.0);
  }

  /** Construct a circle with a specified radius */
  public CircleWithCustomException(double newRadius)
      throws InvalidRadiusException {
    setRadius(newRadius);
    numberOfObjects++;
  }

  /** Return radius */
  public double getRadius() {
    return radius;
  }

  /** Set a new radius */
  public void setRadius(double newRadius)
      throws InvalidRadiusException {
    if (newRadius >= 0)
      radius = newRadius;
    else
      throw new InvalidRadiusException(newRadius);
  }

  /** Return numberOfObjects */
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  public double findArea() {
    return radius * radius * 3.14159;
  }
}
```

Custom Exception Class Example

```
public class TestCircleWithCustomException {
  public static void main(String[] args) {
    try {
      new CircleWithCustomException(5);
      new CircleWithCustomException(-5);
      new CircleWithCustomException(0);
    }
    catch (InvalidRadiusException ex) {
      System.out.println(ex);
    }

    System.out.println("Number of objects created: " +
      CircleWithCustomException.getNumberOfObjects());
  }
}
```

```
InvalidRadiusException: Invalid radius -5.0
Number of objects created: 1
```

Assertions

- An assertion is a Java statement that enables you to assert an assumption about your program. An assertion contains a Boolean expression that should be true during program execution. Assertions can be used to assure program correctness and avoid logic errors.

Declaring Assertions

An *assertion* is declared using the new Java keyword assert in JDK 1.4 as follows:

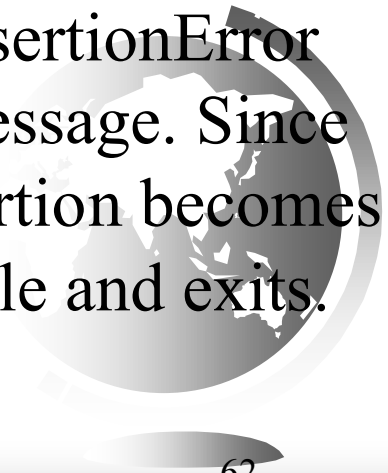
assert *assertion*; or
assert *assertion* : *detailMessage*;

where a**ssert**ion is a Boolean expression and *detailMessage* is a primitive-type or an Object value.

Executing Assertions

When an assertion statement is executed, Java evaluates the assertion. If it is false, an AssertionError will be thrown. The AssertionError class has a no-arg constructor and seven overloaded single-argument constructors of type int, long, float, double, boolean, char, and Object.

For the first assert statement with no detail message, the no-arg constructor of AssertionError is used. For the second assert statement with a detail message, an appropriate AssertionError constructor is used to match the data type of the message. Since AssertionError is a subclass of Error, when an assertion becomes false, the program displays a message on the console and exits.



Executing Assertions Example

```
public class AssertionDemo {
  public static void main(String[] args) {
    int i; int sum = 0;
    for (i = 0; i < 10; i++) {
      sum += i;
    }
    assert i == 10;
    assert sum > 10 && sum < 5 * 10 : "sum is " + sum;
  }
}
```

Compiling Programs with Assertions

Since assert is a new Java keyword introduced in JDK 1.4, you have to compile the program using a JDK 1.4 compiler. Furthermore, you need to include the switch –source 1.4 in the compiler command as follows:

javac –source 1.4 AssertionDemo.java

NOTE: If you use JDK 1.5, there is no need to use the –source 1.4 option in the command.

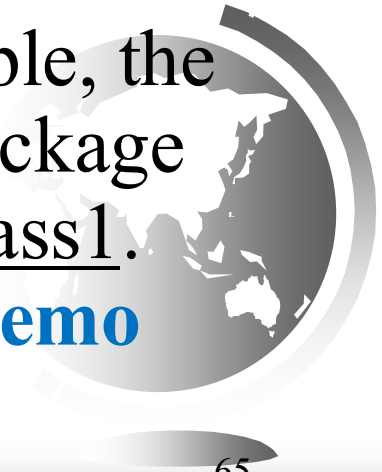
Running Programs with Assertions

By default, the assertions are disabled at runtime. To enable it, use the switch –enableassertions, or –ea for short, as follows:

```
java –ea AssertionDemo
```

Assertions can be selectively enabled or disabled at class level or package level. The disable switch is –disableassertions or –da for short. For example, the following command enables assertions in package package1 and disables assertions in class Class1.

```
java –ea:package1 –da:Class1 AssertionDemo
```

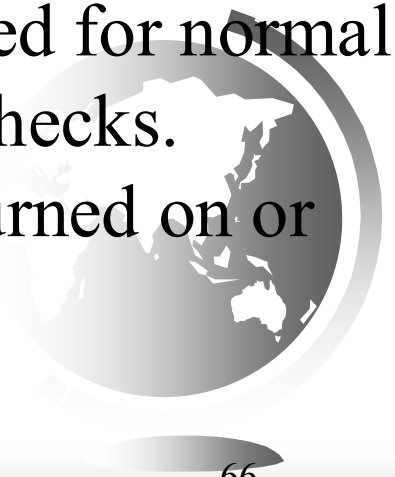


Using Exception Handling or Assertions

Assertion should not be used to replace exception handling.

- Exception handling deals with unusual circumstances during program execution.
- Assertions are to assure the correctness of the program.
- Exception handling addresses robustness and assertion addresses correctness.
- Like exception handling, assertions are not used for normal tests, but for internal consistency and validity checks.

Assertions are checked at runtime and can be turned on or off at startup time.



Using Exception Handling or Assertions, cont.

Do not use assertions for argument checking in public methods. Valid arguments that may be passed to a public method are considered to be part of the method's contract. The contract must always be obeyed whether assertions are enabled or disabled. For example, the following code in the Circle class should be rewritten using exception handling.

```
public void setRadius(double newRadius) {
  assert newRadius >= 0;
  radius =  newRadius;
}
```

Using Exception Handling or Assertions, cont.

Use assertions to reaffirm assumptions. This gives you more confidence to assure correctness of the program. A common use of assertions is to replace assumptions with assertions in the code.

Using Exception Handling or Assertions, cont.

Another good use of assertions is place assertions in a switch statement without a default case. For example,

```
switch (month) {
  case 1: ... ; break;
  case 2: ... ; break;
  ...
  case 12: ... ; break;
  default: assert false : "Invalid month: " + month
}
```

The File Class

- The File class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion.
- The filename is a string.
- For example, **new File("c:\\book")** creates a **File** object for the directory **c:\book**
- The directory separator for Windows is a backslash (\). The backslash is a special character in Java and should be written as \\ in a string literal
- The File class is a wrapper class for the file name and its directory path.

java.io.File	
`+File(pathname: String)`	Creates a `File` object for the specified path name. The path name may be a directory or a file.
`+File(parent: String, child: String)`	Creates a `File` object for the child under the directory parent. The child may be a file name or a subdirectory.
`+File(parent: File, child: String)`	Creates a `File` object for the child under the directory parent. The parent is a `File` object. In the preceding constructor, the parent is a string.
`+exists(): boolean`	Returns true if the file or the directory represented by the `File` object exists.
`+canRead(): boolean`	Returns true if the file represented by the `File` object exists and can be read.
`+canWrite(): boolean`	Returns true if the file represented by the `File` object exists and can be written.
`+isDirectory(): boolean`	Returns true if the `File` object represents a directory.
`+isFile(): boolean`	Returns true if the `File` object represents a file.
`+isAbsolute(): boolean`	Returns true if the `File` object is created using an absolute path name.
`+isHidden(): boolean`	Returns true if the file represented in the `File` object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.
`+getAbsolutePath(): String`	Returns the complete absolute file or directory name represented by the `File` object.
`+getCanonicalPath(): String`	Returns the same as `getAbsolutePath()` except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).
`+getName(): String`	Returns the last name of the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getName()` returns `test.dat`.
`+getPath(): String`	Returns the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getPath()` returns `c:\book\test.dat`.
`+getParent(): String`	Returns the complete parent directory of the current directory or the file represented by the `File` object. For example, new `File("c:\\book\\test.dat").getParent()` returns `c:\book`.
`+lastModified(): long`	Returns the time that the file was last modified.
`+length(): long`	Returns the size of the file, or 0 if it does not exist or if it is a directory.
`+listFile(): File[]`	Returns the files under the directory for a directory `File` object.
`+delete(): boolean`	Deletes the file or directory represented by this `File` object.The method returns true if the deletion succeeds.
`+renameTo(dest: File): boolean`	Renames the file or directory represented by this `File` object to the specified name represented in dest. The method returns true if the operation succeeds.
`+mkdir(): boolean`	Creates a directory represented in this `File` object. Returns true if the the directory is created successfully.
`+mkdirs(): boolean`	Same as `mkdir()` except that it creates directory along with its parent directories if the parent directories do not exist.

Problem: Absolute file names

- An *absolute file name* (or *full name*) contains a file name with its complete path and drive letter.
- **c:\book\Welcome.java** is the absolute file name for the file **Welcome.java** on the Windows operating system.
- On the UNIX platform, the absolute file name maybe **/home/liang/book/Welcome.java**.
- A *relative file name* is in relation to the current working directory. The complete directory path for a relative file name is omitted.
- If you use a file name such as **c:\\book\\Welcome.java**, it will work on Windows but not on other platforms.
- The forward slash (/) is the Java directory separator. The statement **new File("image/us.gif")** works on Windows, UNIX, and any other platform.

Problem: Explore File Properties

```java
public class Exam {
 public static void main(String[] args) {
 java.io.File file = new java.io.File("us.gif");
 System.out.println("Does it exist? " + file.exists());
 System.out.println("The file has " + file.length() + " bytes");
 System.out.println("Can it be read? " + file.canRead());
 System.out.println("Can it be written? " + file.canWrite());
 System.out.println("Is it a directory? " + file.isDirectory());
 System.out.println("Is it a file? " + file.isFile());
 System.out.println("Is it absolute? " + file.isAbsolute());
 System.out.println("Is it hidden? " + file.isHidden());
 System.out.println("Absolute path is " +
 file.getAbsolutePath());
 System.out.println("Last modified on " +
 new java.util.Date(file.lastModified()));
 }
 }
```

Problems | Javadoc | Declaration | Console

<terminated> Exam [Java Application] C:\Program Files\Java\jre1.8.0_191\bin\javaw.exe (Nov 29, 2018, 11:58:14 A

```
Does it exist? false
The file has 0 bytes
Can it be read? false
Can it be written? false
Is it a directory? false
Is it a file? false
Is it absolute? false
Is it hidden? false
Absolute path is C:\Users\Hakan\OneDrive\Courses\cmpe211\EclipseWorkout\Exam\us.gif
Last modified on Thu Jan 01 02:00:00 EET 1970
```

Text I/O

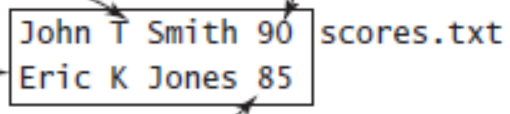
- A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file.
- In order to perform I/O, you need to create objects using appropriate Java I/O classes.
- read/write for strings and numeric values from/to a text file can be done using the **Scanner** and **PrintWriter** classes

Writing Data Using PrintWriter

java.io.PrintWriter	
`+PrintWriter(file: File)`	Creates a `PrintWriter` object for the specified file object.
`+PrintWriter(filename: String)`	Creates a `PrintWriter` object for the specified file-name string.
`+print(s: String): void`	Writes a string to the file.
`+print(c: char): void`	Writes a character to the file.
`+print(cArray: char[]): void`	Writes an array of characters to the file.
`+print(i: int): void`	Writes an `int` value to the file.
`+print(l: long): void`	Writes a `long` value to the file.
`+print(f: float): void`	Writes a `float` value to the file.
`+print(d: double): void`	Writes a `double` value to the file.
`+print(b: boolean): void`	Writes a `boolean` value to the file.
Also contains the overloaded `println` methods.	A `println` method acts like a `print` method; additionally, it prints a line separator. The line-separator string is defined by the system. It is `\r\n` on Windows and `\n` on Unix.
Also contains the overloaded `printf` methods.	The `printf` method was introduced in §4.6, "Formatting Console Output."

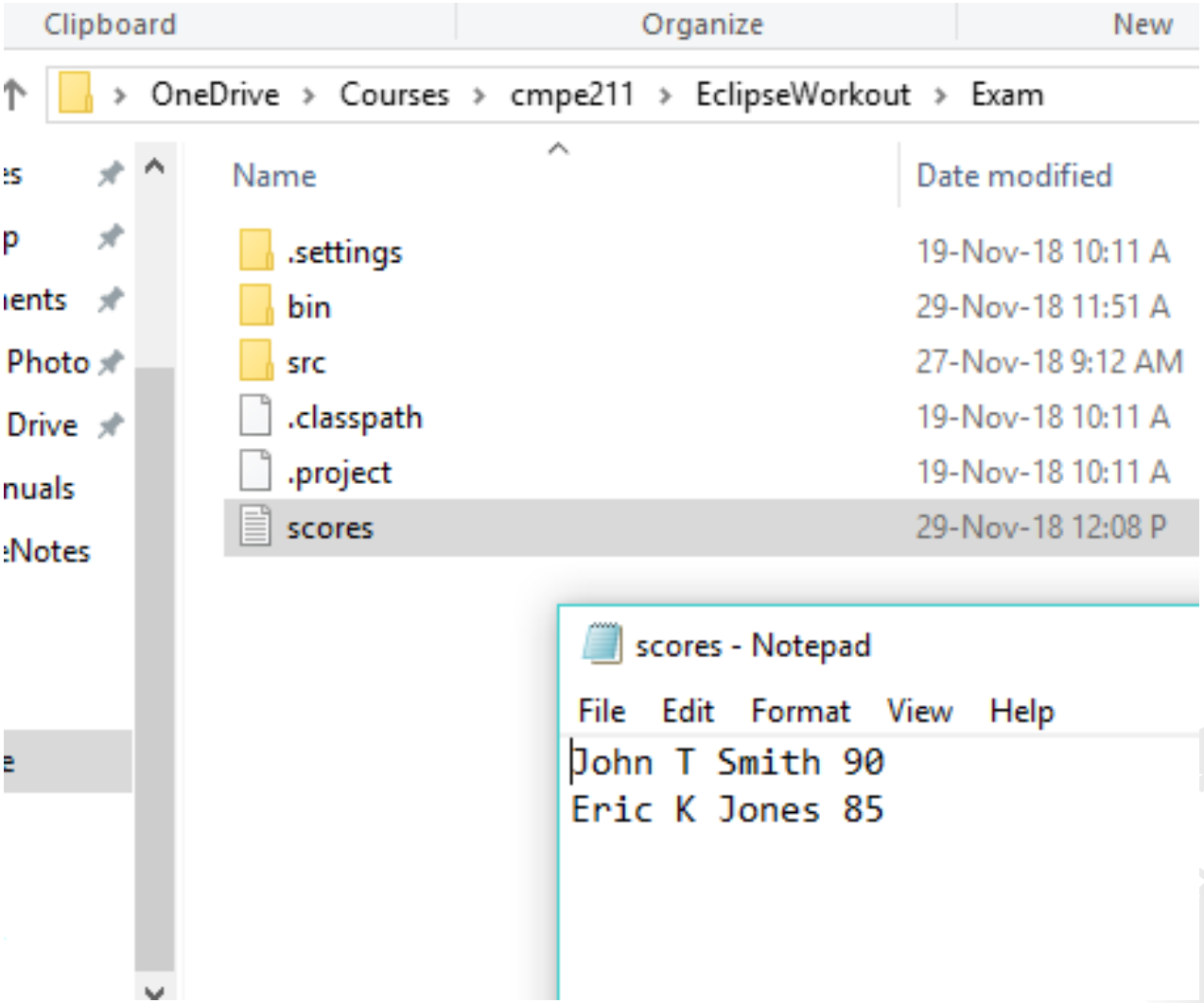
Example

```
public class WriteData {
  public static void main(String[] args) throws IOException {
    java.io.File file = new java.io.File("scores.txt");
    if (file.exists()) {
      System.out.println("File already exists");
      System.exit(1);
    }

    // Create a file
    java.io.PrintWriter output = new java.io.PrintWriter(file);

    // Write formatted output to the file
    output.print("John T Smith ");
    output.println(90);
    output.print("Eric K Jones ");
    output.println(85);

    // Close the file
    output.close();
  }
}
```



Import using “import java.io.IOException”

Example



Reading Data Using Scanner

java.util.Scanner	
`+Scanner(source: File)`	Creates a `Scanner` that scans tokens from the specified file.
`+Scanner(source: String)`	Creates a `Scanner` that scans tokens from the specified string.
`+close()`	Closes this scanner.
`+hasNext(): boolean`	Returns true if this scanner has more data to be read.
`+next(): String`	Returns next token as a string from this scanner.
`+nextLine(): String`	Returns a line ending with the line separator from this scanner.
`+nextByte(): byte`	Returns next token as a `byte` from this scanner.
`+nextShort(): short`	Returns next token as a `short` from this scanner.
`+nextInt(): int`	Returns next token as an `int` from this scanner.
`+nextLong(): long`	Returns next token as a `long` from this scanner.
`+nextFloat(): float`	Returns next token as a `float` from this scanner.
`+nextDouble(): double`	Returns next token as a `double` from this scanner.
`+useDelimiter(pattern: String): Scanner`	Sets this scanner's delimiting pattern and returns this scanner.

Reading Data Using Scanner

```
import java.util.Scanner;

public class ReadData {
  public static void main(String[] args) throws Exception {
    // Create a File instance
    java.io.File file = new java.io.File("scores.txt");

    // Create a Scanner for the file
    Scanner input = new Scanner(file);

    // Read data from a file
    while (input.hasNext()) {
      String firstName = input.next();
      String mi = input.next();
      String lastName = input.next();
      int score = input.nextInt();
      System.out.println(
        firstName + " " + mi + " " + lastName + " " + score);
    }

    // Close the file
    input.close();
  }
}
```